

Program Structures and Algorithms
Spring 2025

NAME: Xinduo Fan

NUID: 002059620

GITHUB LINK: <https://github.com/XDagain-jpg/DSAIPG>

Assignment 1 Task:

1. Implement 4-ary heap with an option of using Floyd's trick or not;
2. use benchmarking (*Benchmark_Timer* class) to compare the following implementations: Basic binary heap; Same but with Floyd's trick (i.e. using the *snake* method); 4-ary heap; Same but with Floyd's trick; Fibonacci Heap.

Code Screenshots:

For *PriorityQueue_4ary.java* which **extends** *PriorityQueue*:

Adjustments are in the following methods, and additional field *maxPrioritySpilled*

```
public PriorityQueue_4ary(Collection<K> ks, Comparator<K> comparator) {
    this(ks.size(), comparator);
    int i = 0;
    for (K k : ks) binHeap[i++] = k;
    m = ks.size();
    int k = (m + 1) / 4 - 1;
    for (; k >= 0; k--) sink(k);
}

private int doHeapify(int k, BiPredicate<Integer, Integer> p) {
    int i = k;
    while (true) {
        int firstChild = firstChild(i);
        if (!(firstChild <= m + first - 1)) break;
        int j = firstChild;

        //4-ary iteration through 4 children
        for (int x = 1; x <= 3; x++) {
            int nextChild = firstChild + x;
            if (nextChild <= m + first - 1 && inverted(j, nextChild))
                j = nextChild;
        }

        if (p.test(i, j)) break;
        swap(i, j);
        i = j;
    }
    return i;
}
```

```

/**
 * Get the index of the parent of the element at index k
 */
private int parent(int k) {
    if (k <= first) return -1; // Prevents negative indices
    return (k - first - 1) / 4 + first;
}

/**
 * Get the index of the first child of the element at index k.
 * The index of the second child will be one greater than the result.
 */
private int firstChild(int k) {
    //return (k + 1 - first) * 2 + first - 1;
    return 4 * (k - first) + 1 + first;
}

private K maxPrioritySpilled = null;

public K getMaxPrioritySpilled() {
    return maxPrioritySpilled;
}

/**
 * Adds a key to the priority queue. If the priority queue is at its capacity,
 * the last element is removed to make space for the new key.
 * CONSIDER we can prevent the new element displacing a higher-priority element but at the cost of an extra
 * Is that worth it?
 *
 * @param key the element to be inserted into the priority queue
 */
public void give(K key) {
    if (m == binHeap.length - first) {
        K discarded = binHeap[m + first - 1];

        // Update maxPrioritySpilled
        if (maxPrioritySpilled == null || comparator.compare(discarded, maxPrioritySpilled) > 0) {
            maxPrioritySpilled = discarded;
        }

        m--;
    }
    binHeap[++m + first - 1] = key;
    swimUp(m + first - 1);
}

```

For its corresponding tests, we also adjust some features to test its structure. Please see the repo for more details.

For the benchmark:

```
public class HeapBenchmark_Assignment4 {

    public static void main(String[] args) {
        int M = 4095; // Initial heap size
        int insertions = 16000;
        int removals = 4000;
        int numTests = 5; // Run the test 5 times, doubling each time

        System.out.println("HeapType,M,Insertions,Removals,Time(ms),MaxPrioritySpilled");

        for (int i = 0; i < numTests; i++) {
            runBenchmark("BinaryHeap", M, insertions, removals, false, false);
            runBenchmark("BinaryHeap-Floyd", M, insertions, removals, false, true);
            runBenchmark("4aryHeap", M, insertions, removals, true, false);
            runBenchmark("4aryHeap-Floyd", M, insertions, removals, true, true);

            // Double values for next test
            M *= 2;
            insertions *= 2;
            removals *= 2;
        }

        System.out.println("Benchmark completed.");
    }

    private static void runBenchmark(String heapType, int M, int insertions, int removals, boolean is4ary, boolean floyd) {
        Random random = new Random();

        PriorityQueue<Integer> pq = createPriorityQueue(M, is4ary, floyd);

        Benchmark_Timer<PriorityQueue<Integer>> benchmark = new Benchmark_Timer<>(
            heapType,
            heap -> {
                for (int i = 0; i < insertions; i++)
                    heap.give(random.nextInt(1000000));

                for (int i = 0; i < removals; i++) {
                    if (!heap.isEmpty())
                        try {
                            heap.take();
                        } catch (PQException e) {
                            e.printStackTrace();
                        }
                }
            }
        );

        double timeMs = benchmark.runFromSupplier(() -> pq, 10);

        System.out.printf("%s,%d,%d,%d,%.3f,%d%n",
            heapType, M, insertions, removals, timeMs, pq.getMaxPrioritySpilled());
    }

    private static PriorityQueue<Integer> createPriorityQueue(int M, boolean is4ary, boolean floyd) {
        if (is4ary) {
            return new PriorityQueue_4ary<Integer>(M, true, Comparator.naturalOrder(), floyd);
        } else {
            return new PriorityQueue<Integer>(M, true, Comparator.naturalOrder(), floyd);
        }
    }
}
```

Unit Test Screenshots:

Finished after 0.032 seconds

Runs: 30/30  Errors: 0  Failures: 0

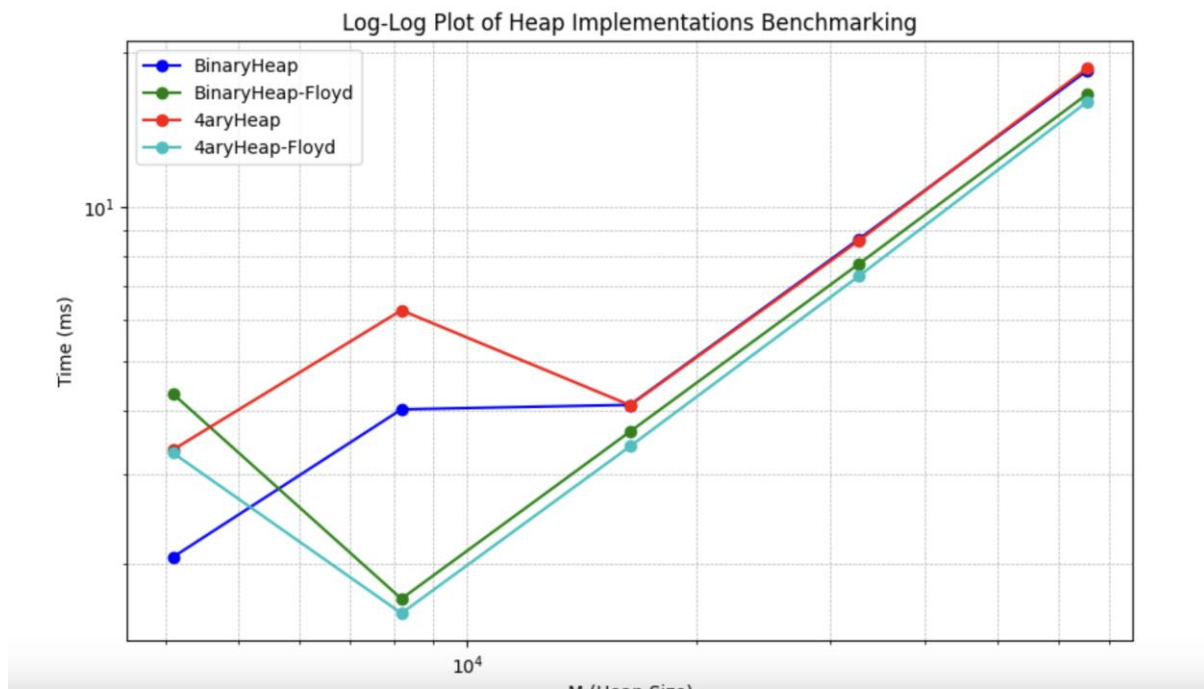
- ✓  com.phasmidsoftware.dsaipg.adt.pq.PriorityQueueTest [Runner: JUnit 4] (0.020 s)
 - ✓  iterator0 (0.000 s)
 - ✓  iterator1 (0.000 s)
 - ✓  testInverted2 (0.008 s)
 - ✓  testGive1 (0.000 s)
 - ✓  testGive2 (0.001 s)
 - ✓  testSink1 (0.001 s)
 - ✓  testTake1 (0.001 s)
 - ✓  testTake2 (0.000 s)
 - ✓  testTake3 (0.000 s)
 - ✓  testTake4 (0.001 s)
 - ✓  testTake5 (0.000 s)
 - ✓  testTake6 (0.000 s)
 - ✓  size (0.000 s)
 - ✓  doTake01 (0.000 s)
 - ✓  doTake02 (0.000 s)
 - ✓  doTake11 (0.001 s)

The following test for 4-ary heap is modified for 4-ary's structure. See the code where we add comments in line explaining the changes.

Runs: 30/30  Errors: 0  Failures: 0

- ✓  com.phasmidsoftware.dsaipg.adt.pq.PriorityQueueTest_4ary [Runner: JUnit 4] (0.017 s)
 - ✓  iterator0 (0.000 s)
 - ✓  iterator1 (0.000 s)
 - ✓  testInverted2 (0.005 s)
 - ✓  testGive1 (0.000 s)
 - ✓  testGive2 (0.001 s)
 - ✓  testSink1 (0.001 s)
 - ✓  testTake1 (0.001 s)
 - ✓  testTake2 (0.000 s)
 - ✓  testTake3 (0.000 s)
 - ✓  testTake4 (0.001 s)
 - ✓  testTake5 (0.000 s)
 - ✓  testTake6 (0.000 s)
 - ✓  testPriorityQueue_4aryList (0.001 s)
 - ✓  size (0.000 s)
 - ✓  doTake01 (0.000 s)
 - ✓  doTake02 (0.001 s)

Observations:



in the plot we can see that BinaryHeap-Floyd and 4aryHeap-Floyd are the fastest implementations, both performing consistently better than their corresponding heap structure without the Floyd's trick. 4-ary heap perform slightly better than their corresponding binary heap implementations when M gets large: when M is smaller, the extra per-node operations in 4-ary heaps make them slightly slower, but they scale better than binary heaps.

Conclusion:

When M is smaller, we should choose binary heap with Floyd's trick, and when M gets bigger, and the heap gets deeper, we should choose 4-ary heap with Floyd's trick.